National University of Computer and Emerging Sciences, Lahore Campus

Course:	Design and Analysis of Algorithms	Course Code:	CS-2009
Program:	BS (CS, DS)	Semester:	Spring 2024
Due Date:	Sep 14, 2025	Total Marks:	45
Sections:	CS-5(C,D), DS-5(B,C)		
Exam:	Assignment#1		

Instruction/Notes:

Instructions: Plagiarism in any form will not be tolerated. Only handwritten, hard copy assignments will be accepted.

Q#1: Solve the recurrence. (Marks: 3*8 = 24)

Use a recursion tree method to determine a good asymptotic upper bound on following recurrences. Please refer to the Appendix of your textbook for using arithmetic, geometric and harmonic series. Clearly show your working and don't forget to mention the relevant formula before writing the final answer.

Part(a): Linear Decay (Attempt any two)

a) $T(N) = 3T(N-1) + \Theta(1)$
b) $T(N) = T(N-2) + O(N^2)$
c) $T(N) = 2T(N-1) + \Theta(N)$
d) $T(N) = T(N-1) + O(\frac{1}{N})$

Part(b): Exponential decay with balanced split of data (Attempt any four)

e) $T(N) = 2T\left(\frac{N}{4}\right) + O(logN)$
f) $T(N) = 10T\left(\frac{N}{2}\right) + \Theta(1)$
g) $T(N) = 2T\left(\frac{N}{2}\right) + O(\frac{N}{logN})$
h) $T(N) = 4T\left(\frac{N}{2}\right) + O(N^2\sqrt{N})$
i) $T(N) = 2T\left(\frac{N}{4}\right) + O(\sqrt{N})$
j) $T(N) = 3T\left(\frac{N}{2}\right) + O(N^3)$
k) $T(N) = 7T\left(\frac{N}{5}\right) + \Theta(1)$
l) $T(N) = 3T\left(\frac{2N}{3}\right) + O(1)$
m) $T(N) = 3T\left(\frac{4N}{5}\right) + O(1)$

Part(c): Exponential decay with unbalanced split of data (Attempt any two)

n) $T(N) = T\left(\frac{N}{4}\right) + T\left(\frac{3N}{4}\right) + O(N^2)$
o) $T(N) = T\left(\frac{3N}{10}\right) + T\left(\frac{7N}{10}\right) + O(N)$
p) $T(N) = T\left(\frac{2N}{7}\right) + T\left(\frac{5N}{7}\right) + O(N^2)$

Q#2: Derive the recurrence of following algorithms. (Marks: 3*5 = 15)

Part(a)
```
Split Data (Arr[], left, right){
  If (left < right) {
    Split Data (Arr, left, right-1)
    Split Data (Arr, left+1, right)
    View Data (Arr, left, right)
  }
}
View Data (Arr[], left, right){
  i=left
  N_Arr[left+right]
  While (i < right) {
    j = left
    k = 1
    while (j < right) {
      N_Arr[k] = Arr[j]
      print Arr[i]
      k = k+1
      j = j*2
    }
    i++
  }
} //initially left = 1 and right = N
```

Part(b)
```
Temp(A[], B[], N){
  if(N==1)
    return
  Temp(A+N/2,B, N/2) + Temp(A, B + N/2, N/2)
  Temp(A+N/2,B, N/2) + Temp(A, B + N/2, N/2)
  Temp(A+N/2,B, N/2) + Temp(A, B + N/2, N/2)
  Temp(A+N/2,B, N/2) + Temp(A, B + N/2, N/2)
  Temp(A+N/2,B, N/2) + Temp(A, B + N/2, N/2)
  Temp(A+N/2,B, N/2) + Temp(A, B + N/2, N/2)
  Temp(A+N/2,B, N/2) + Temp(A, B + N/2, N/2)
  Temp(A+N/2,B, N/2) + Temp(A, B + N/2, N/2)
  Solve(A,B,N)
}
Solve (A[], B[], N){
  for(i=1 to N){
    for(j=1 to i){
      print A[i]*B[j]
    }
  }
}
```

Part (c):
```
Mystery (N){
  If (N > 1){
    Print "Deriving recurrence is fun"
    Mystery (2N⁄3)
    for (i=1 to N)
      Print "Solving recurrence is fun"
    Mystery (N⁄5)
  }
}
```

Part(d):
```
Stooge-Sort (Arr [], left, right){
  if (Arr[left] > A[right])
    exchange(A[left], A[right])
  if (left > right)
    return
  else{
    third = (right - left + 1)/3
    Stooge-Sort (Arr, left, right - third)
    Stooge-Sort (Arr, left + third, right)
    Stooge-Sort (Arr, left, right - third)
  }
}
```
Also provide the reason that how these recursive calls will sort the array and why we need the third recursive call with data ranging from Left to 3rd quarter of the array.

Part (e):
```
data_partition(A[], left, right, N){
  if(left < right) {
    k = 2*((right-left+1)/7)
    v1 = data_partition(A, left, k, k)
    v2 = data_partition(A, k+1, right, N)
    v3 = update(A, left, right)
    return v1+v2+v3
  }
  return A[right]
}
```

```
update(A[], left, right){
  i = left
  r = 0
  for(i=left; i<=right; i*=2){
    for(m=left; m<=i; m++){
      A[m] = A[i]*A[m]
      r = A[m]
    }
  }
  return r
}
```

Q#3: Asymptotic Analysis (Marks: 1*5 = 5)

a) Prove that $T(N) = \Theta(g(N))$ by finding the appropriate g(N) and constants (n_0, c_1 and c_2)

$$f(N) = (1000)2^N + 4^N$$

b) Prove or disprove the following statement

$$2^{N+12} \text{ is } O(2^N)$$

c) Prove or disprove the following statement

$$4^{12N} \text{ is } O(64^N)$$

d) Prove that $T(N) = \Theta(N^3)$ by finding appropriate constants (n_0, c_1 and c_2).

$$T(N) = \frac{1}{8} N^3 - 5N^2$$

e) Prove that $T(N) = \Theta(g(N))$ by finding the appropriate g(N) and constants.

$$T(N) = 5\sqrt{N} + 3N^2 logN + \frac{N}{(logN)^2}$$

f) Arrange the functions $(1.5)^N, n^{100}, (logN)^3, \sqrt{N}\, logN, 10^N, (n!), logN$ in a list such that each function is providing the upper bound (big-Oh) of previous one.